

SAIDL CORE ML

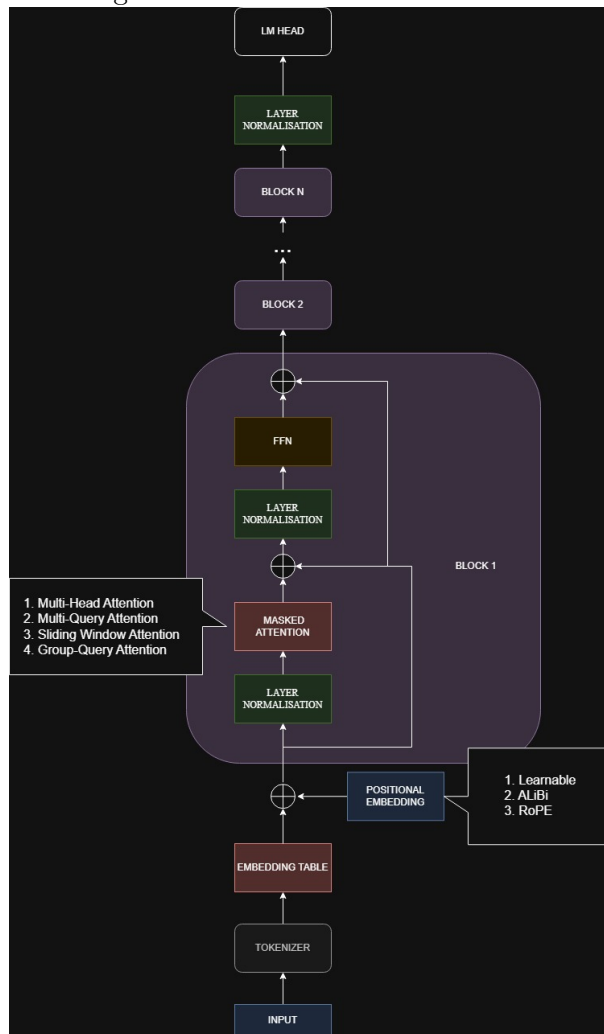
Transformer Implementation

Shivam Bamba
2025B4PS1402G

1 Overview

In this section, I discuss my Transformer implementation. Figure 1 shows an overview of the architecture implemented for my core ML task. Below, I detail all the architectural types and components incorporated into my code.

Figure 1: Transformer Architecture



2 Experimental Setup

I used a single A100 GPU for my final run and utilized Kaggle notebooks for testing and debugging. I also used GitHub Copilot's autocomplete feature for certain parts of my code. To manage configurations, I used **Hydra**, and to visualize and log all experiments, I used **Weights & Biases (WandB)**.

For all my experiments, I fixed the following hyperparameters:

- **Hidden Size:** 512
- **Block Size:** 512, 1024, 2048
- **Number of Layers:** 4
- **Attention Dropout Probability:** 0.1
- **Optimizer:** Adam
- **Learning Rate (Lr):** $1 \times 10^{-3}, 1 \times 10^{-4}$
- **Epochs:** 5, 10
- **Batch Size:** 4
- **Eval Batch Size:** 8

Types of Attention Mechanisms

- **Multi-Head Attention:** Number of heads = 8
- **Multi-Query Attention:** Number of queries = 8; Number of keys & values = 1
- **Sliding Window Attention:** Window size = 512
- **Grouped-Query Attention:** Number of queries = 8; Number of keys & values = 4 (Number of queries per key & value = 2)

Types of Positional Embedding

- Learnable
- ALiBi
- RoPE (Base Value = 1024, 2048, 10000)

3 Attention Mechanisms

3.1 Multi-Head Attention (MHA)

What it does: MHA lets every token attend to every other token across its parallel heads. Each head has its own Query, Key, and Value vectors. Having these many parameters to train allows the model to learn better representations.

Formula:

$$\text{Head Size } (d) = \frac{\text{Hidden Size}}{\text{Number of Heads}} \quad (1)$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V \quad (2)$$

KV-cache size per layer: Number of heads $\times d \times 2$

Output: Tensor of shape (B, T, C) — same shape as the input.

Analysis: Initially, I created two distinct modules named `Head` and `MultiHead`, running a `for` loop in the `MultiHead` module for every head. However, the throughput was very low. Subsequently, I utilized vectorization, which significantly increased the training throughput from **36,978.53 tokens/sec** to **45,206.08 tokens/sec** — a **22.22% relative improvement**.

3.2 Multi-Query Attention (MQA)

What it does: MQA uses a single shared K and V projection for all query heads. This slashes the KV-cache from H copies down to 1 copy per layer, greatly reducing memory bandwidth.

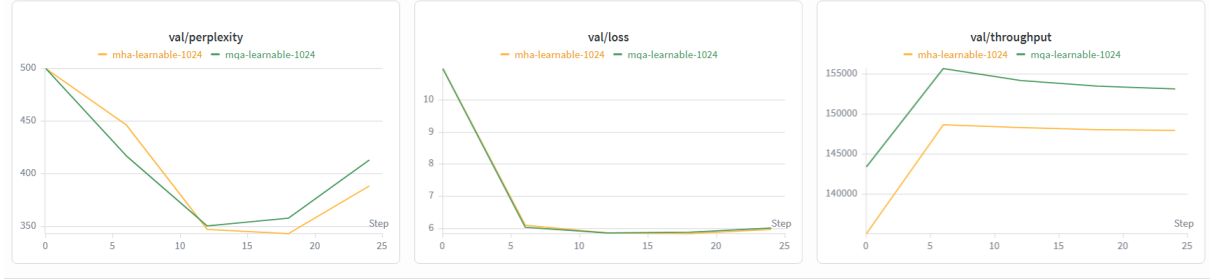
Formula: The attention formula remains the same; we simply broadcast the same K and V to each head.

$$\text{KV-cache size per layer} = 1 \times d \times 2 \quad (3)$$

Output: Tensor of shape (B, T, C) — same shape as the input.

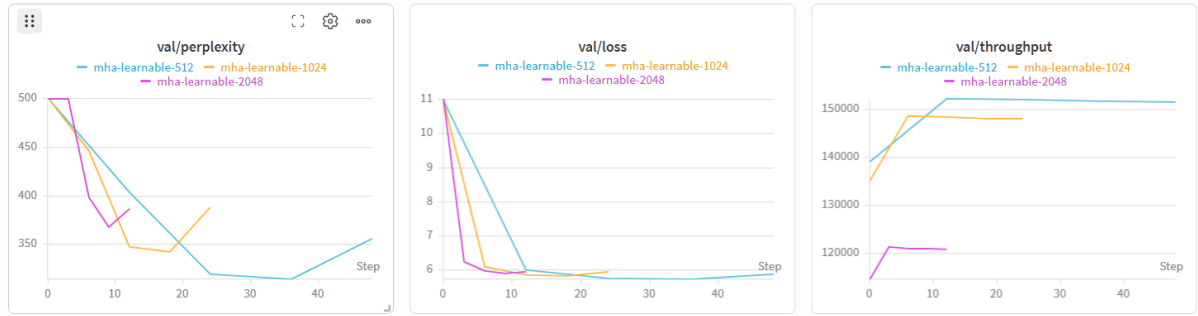
Analysis:

Figure 2: I observed that the throughput is higher for MQA, but MHA achieves better performance as indicated by its lower perplexity.



3.3 Effect of Context Length

Figure 3: I used MHA with learnable embeddings for three context lengths: 512, 1024, and 2048. The throughput for the 512 context length is higher, which is expected since attention complexity is quadratic. However, it achieving the lowest perplexity was unexpected; this could be because it ran for a greater number of steps over the same number of epochs.



3.4 Grouped Query Attention (GQA)

What it does: GQA sits between MHA and MQA. Heads are divided into G groups; every head within a group shares one K/V projection. Setting `group_size = H` collapses the mechanism to MQA, whereas `group_size = 1` reverts it to full MHA.

Formula: The attention formula remains the same, broadcasting the group's K and V to each head within the group.

$$\text{Group size} = G \Rightarrow \text{KV-cache size per layer} = G \times d \times 2 \quad (4)$$

Output: Tensor of shape (B, T, C) — same shape as the input.

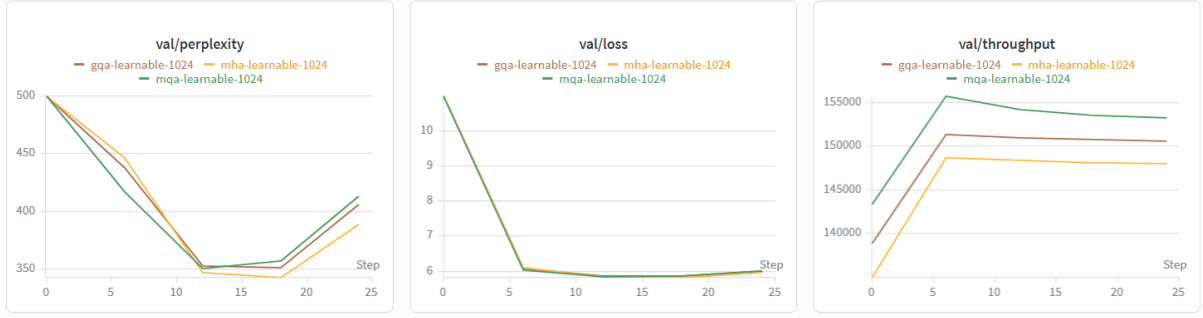
Analysis:

4 Positional Embeddings

4.1 Learnable Embedding

What it does: Transformers process all tokens concurrently, meaning they have no inherent understanding of sequence order or word positions. Learnable Positional Embeddings solve this by assigning a dedicated, trainable vector to every absolute position in a sequence.

Figure 4: I observed that the throughput sits between that of MQA and MHA. However, among all three, MHA still yields the best performance due to its lower perplexity.



Output: X with a shape of (B, T, C) . Every token's vector now contains a fused representation of both its meaning and its location in the sequence.

4.2 Attention with Linear Biases (ALiBi)

What it does: ALiBi adds a fixed, parameter-free bias to attention scores before the softmax operation. The bias is proportional to the distance between query and key positions. Each head has a different slope, so the model naturally develops heads that specialize in short- versus long-range contexts.

Formula:

$$\text{Slope}_h = \frac{1}{2^{h/H}} \quad \text{for } h = 0, \dots, H - 1 \quad (5)$$

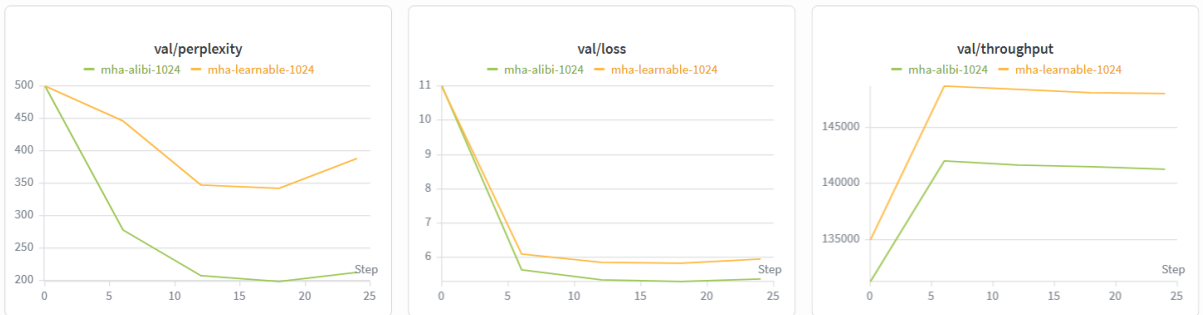
$$\text{Dist}[i, j] = i - j \quad (\text{relative distances}) \quad (6)$$

$$\text{ALiBi_bias}[h, i, j] = \text{Slope}_h \times \text{Dist}[i, j] \quad (7)$$

Output: `alibi_bias` of shape (H, T, T) — a per-head additive bias on raw attention scores. No position embeddings are added directly to the token vectors.

Analysis:

Figure 5: The perplexity for the ALiBi model decreased significantly, but the throughput also decreased due to the addition of extra parameters.



4.3 Rotary Position Embedding (RoPE)

What it does: RoPE encodes position by rotating Q and K vectors rather than adding a learned embedding to token vectors. Because the inner product $Q_m \cdot K_n$ becomes a function solely of the relative offset $(m - n)$, the model naturally learns relative distances.

Formula:

$$\text{inv_freq}_i = \frac{1}{\text{base}^{2i/\text{dim}}} \quad \text{for } i = 0, \dots, \frac{\text{dim}}{2} - 1 \quad (8)$$

$$\text{freqs}[\text{pos}] = \text{pos} \times \text{inv_freq} \quad (9)$$

$$\text{rotate_half}(x) = \text{concat} \left(-x \left[\dots, \frac{\text{dim}}{2} : \right], x \left[\dots, : \frac{\text{dim}}{2} \right] \right) \quad (10)$$

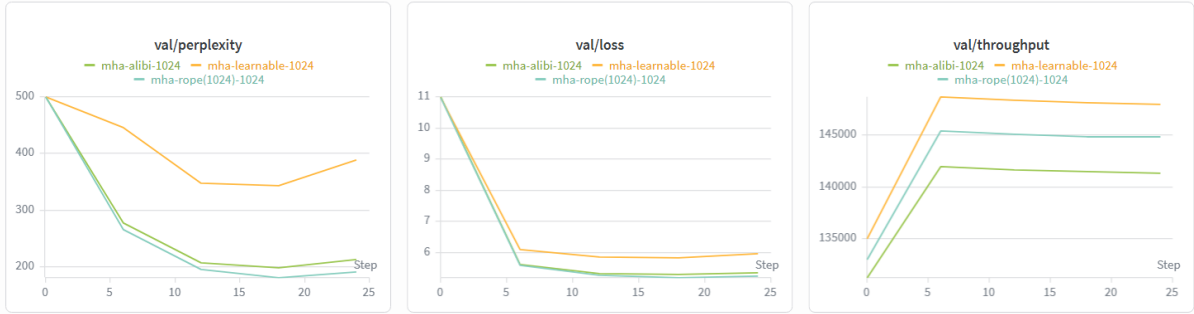
$$q_{\text{rotated}} = q \odot \cos(\text{freqs}) + \text{rotate_half}(q) \odot \sin(\text{freqs}) \quad (11)$$

$$k_{\text{rotated}} = k \odot \cos(\text{freqs}) + \text{rotate_half}(k) \odot \sin(\text{freqs}) \quad (12)$$

Output: $q_{\text{rotated}}, k_{\text{rotated}}$ of shape (B, H, T, d) — the same shape as the input q, k .

Analysis:

Figure 6: RoPE achieves the best perplexity among all three methods and maintains a throughput that falls between the other two.

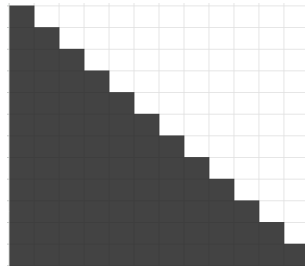


5 Attention Mask

5.1 Causal Attention Mask

What it does: A full causal attention mask allows each query to attend to all past tokens. It restricts each query from looking at future tokens, ensuring there is no information leakage.

Figure 7: Causal attention: The black area represents the active region, while the white area represents the masked (dead) region that cannot be seen by the current query.



5.2 Sliding Window Attention Mask

What it does: Full causal attention requires $\mathcal{O}(T^2)$ in memory and compute. Sliding Window Attention caps each query to attend to only the most recent W tokens, making the cost $\mathcal{O}(T \times W)$ — linear in T for a fixed window size W .

Figure 8: Sliding window attention: The black area represents the active region, while the white area represents the masked (dead) region that cannot be seen by the current query.

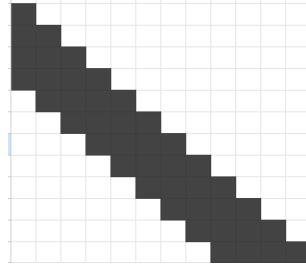
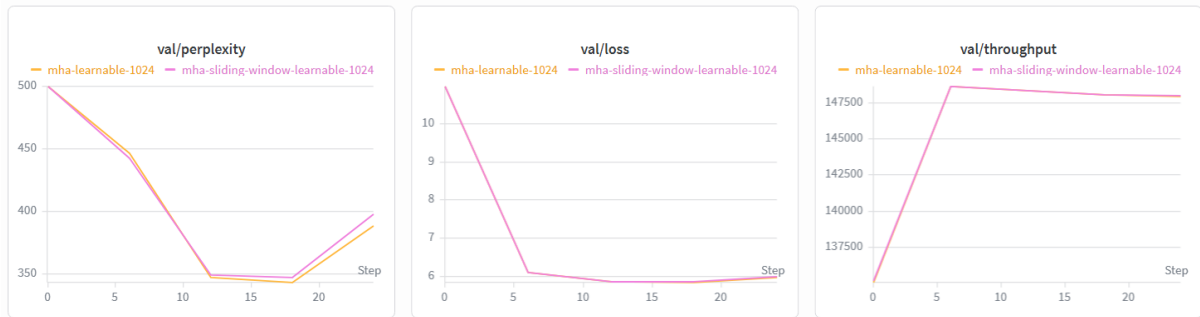


Figure 9: MHA with a sliding window mask performs slightly worse than MHA with a causal attention mask. The throughput of the sliding window mask should ideally be higher than that of the causal mask, but my current implementation is naive; I plan to optimize it in the future.

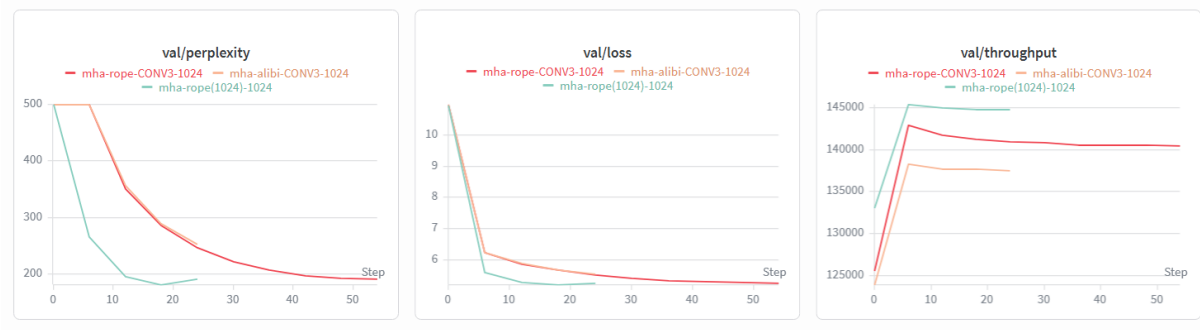


6 Convolution + Attention Hybrids

What it does: I implemented 3-gram context convolution. Initially, I implemented this naively, which allowed the queries to attend to future tokens, introducing data leakage. While this resulted in low perplexity, it was methodologically incorrect. Later, I introduced a padding layer that prevented the queries from seeing future tokens, effectively stopping the leakage.

Analysis:

Figure 10: Convolution (Best+Best)



7 Other Transformer Components

7.1 Feed-Forward Block (FFN)

What it does: The FFN is the per-token non-linearity in every Transformer Block. Applied independently at each position, it acts as a key-value memory storing factual and pattern information. Expanding the embedding dimension to $n \times C$ (where $n = 4$) and then compressing

it back introduces the capacity to learn complex transformations without the quadratic cost of attention.

Formula:

$$\text{FFN}(x) = \text{Dropout}(W_2 \cdot \text{ReLU}(W_1 x + b_1) + b_2) \quad (13)$$

Output: (B, T, C) — the same shape as the input.

7.2 Transformer Block

What it does: Each block contains: Layer Normalization $\rightarrow$ Masked Attention $\rightarrow$ Layer Normalization $\rightarrow$ Feed-Forward Block. Residual connections allow gradients to bypass each sub-layer during backpropagation, enabling the training of dozens of layers.

Formula:

$$x = x + \text{Attention}(\text{LayerNorm}_1(x), \text{mask}) \quad (14)$$

$$x = x + \text{FFN}(\text{LayerNorm}_2(x)) \quad (15)$$

Output: (B, T, C) — the same shape as the input.

7.3 LayerNorm

What it does: LayerNorm normalizes the internal representations of a single token across its embedding dimension so that they have a mean of zero and a variance of one. It then scales and shifts this result using learnable parameters (γ, β) . This is crucial because it stabilizes gradients during training, preventing them from vanishing or exploding in deep networks.

Formula:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (16)$$

Output: (B, T, C) — the same shape as the input. Every token is independently normalized across its feature dimension.

7.4 Tokenizer

What it does: Transformers cannot process raw strings; they require numbers. The tokenizer acts as the bridge between human language and machine logic by converting raw text into sequences of discrete integers (token IDs). I used the **GPT-2 Tokenizer**.

Output: `idx` with shape (B, T) .

7.5 Embedding Table

What it does: Once the text is tokenized into integers, the Embedding Table translates those discrete token IDs into dense, continuous mathematical vectors. It acts as a massive lookup matrix where each row corresponds to a specific vocabulary token and contains a sequence of learnable weights.

Output: X with shape (B, T, C) . It replaces every integer token ID with its corresponding C -dimensional vector of learned features.

7.6 Hyperparameter Search

What I did: I performed a grid search on two parameters: learning rate and the number of epochs. I found that with a learning rate of 1×10^{-3} , the model overfits after 2 epochs. With a learning rate of 1×10^{-4} , the model continues training even after 10 epochs but still cannot beat the performance of the 1×10^{-3} model. To save compute resources, I trained my final models for 5 epochs using a learning rate of 1×10^{-3} .

Figure 11: GridSearch



8 Logging and Reporting

Conv	Attn	Cxt	Mask	Pos Embed	T. Loss	V. Loss	V. PPL	GPU	T. Time	Throughput
No	MQA	1024	Causal	learnable	2.06	5.85	350.49	9.99	51.81	155731.23
No	MHA	1024	Sliding	learnable	3.98	5.85	347.29	10.07	53.70	148682.08
No	MHA	1024	Causal	RoPE	3.48	5.20	182.33	10.09	54.65	145426.36
No	MHA	2048	Causal	learnable	4.29	5.98	368.58	17.57	59.59	121346.57
No	MHA	1024	Causal	learnable	3.99	5.83	343.09	10.07	53.71	148660.59
No	MHA	512	Causal	learnable	3.82	5.75	315.06	6.10	50.98	152205.61
No	MHA	1024	Causal	ALiBi	3.54	5.29	199.24	10.09	54.47	142003.39
No	GQA	1024	Causal	learnable	3.99	5.86	351.33	10.05	52.66	151354.58
Yes	MHA	1024	Causal	RoPE	3.99	5.25	191.45	10.14	56.32	142958.77

Table 1: Transformer Configuration Experiments

Conv: Convolution, Attn: Attention, Cxt: Context length, Pos Embed: Positional Embedding, T. Loss: Training loss, V. Loss: Validation Loss, V. PPL: Validation Perplexity, GPU: Peak GPU memory usage (GB), T. Time: Training time per epoch (sec).

9 Materials Studied

I started with Andrew Ng’s *Machine Learning Specialization* on Coursera. After that, I watched four lectures of CS231n to get the gist of how batch normalization and related concepts work. I then completed Andrej Karpathy’s YouTube playlist, *Neural Networks: Zero to Hero*, which significantly helped me understand how Transformers and backpropagation work, along with the underlying mathematics. For attention variants, I used a DataMListic video to understand how different types operate. Following that, I watched Yannic Kilcher’s ALiBi paper explanation video, which was instrumental in helping me grasp the mechanism and math behind it. Finally, I watched a Vizura video for RoPE.